PROJET DE SESSION NLP

Chatbot Assistant Service Client Intelligent basé sur le RAG

(Retrieval-Augmented Generation)

Generative AI / NLP / Full-Stack :

El Mehdi LAKHIAL

2025/03/04

1.1 Contexte Général

Dans les environnements d'entreprise modernes, la gestion et l'accès rapide à l'information interne constituent un enjeu stratégique majeur. Les équipes techniques, notamment DevOps, support et développement, manipulent quotidiennement une grande quantité de documents : procédures internes, guides techniques, politiques de sécurité, configurations d'infrastructure, rapports d'incident et documentation métier.

Cependant, ces informations sont souvent :

- Dispersées dans plusieurs formats (**PDF, TXT, DOCX**)
- Stockées dans différents emplacements
- Difficiles à rechercher efficacement
- Soumises à des règles strictes de confidentialité

Dans une organisation hiérarchisée, toutes les données ne sont pas accessibles à tous les utilisateurs. Un administrateur peut accéder à l'ensemble des ressources, tandis qu'un employé d'une filiale ou d'un département ne doit consulter que les documents autorisés pour son rôle.

Les moteurs de recherche classiques par mots-clés ne suffisent pas dans ce contexte, car :

- Ils ne comprennent pas le sens des requêtes.
- Ils ne tiennent pas compte du contexte métier.
- Ils ne gèrent pas nativement les restrictions hiérarchiques complexes.

L'émergence des modèles de langage avancés **(LLMs)** combinés aux architectures RAG offre une solution permettant :

- Une recherche sémantique intelligente.
- Une génération de réponses contextualisées.
- Un contrôle fin des accès basé sur les rôles.

Ce projet s'inscrit dans cette perspective : concevoir un système intelligent interne capable de structurer, sécuriser et accélérer l'accès aux connaissances organisationnelles.

1.2 Problématique

L'intégration d'un LLM brut dans un système d'entreprise pose plusieurs risques :

1. **Hallucinations :** génération d'informations incorrectes.
2. **Absence de contrôle d'accès :** risque d'exposition de données sensibles.
3. **Non-traçabilité** des sources.
4. **Non-prise** en compte de la structure hiérarchique de l'organisation.

Dans un environnement hiérarchisé **(Admin, leaders, employee)**, la problématique devient plus complexe :

Comment concevoir un système intelligent capable de :

- Fournir des réponses précises basées uniquement sur la documentation interne ?

- Respecter strictement les règles d'accès hiérarchiques ?
- Garantir que chaque utilisateur ne consulte que les informations autorisées ?
- Optimiser la circulation rapide des données au sein de l'organisation ?

Autrement dit :

Comment implémenter **une architecture RAG sécurisée**, **multi-rôles et hiérarchique**, adaptée aux besoins internes d'une entreprise technique, tout en maintenant performance, cohérence et confidentialité des données ?

1.3 Valeur Ajoutée

L'implémentation de cette architecture RAG hiérarchique apporte une valeur stratégique interne, centrée sur la gestion intelligente et sécurisée des connaissances organisationnelles.

Pour l'Administration (Admin)

- Centralisation structurée des connaissances internes (procédures, guides techniques, configurations, politiques).
- Contrôle d'accès hiérarchique fin basé sur les rôles.
- Réduction du temps de recherche documentaire.
- Meilleure gouvernance des données internes.
- Sécurisation des informations sensibles grâce au filtrage par rôle au niveau du moteur de retrieval.

Pour les équipes (Admin / Leaders / employe)

- Accès rapide et sémantique à la documentation autorisée.
- Réduction du temps passé à chercher dans des fichiers volumineux.
- Assistance intelligente adaptée au contexte métier.
- Amélioration de la productivité des équipes techniques (DevOps, support, développement).

Le système ne se limite pas à générer des réponses ; il agit comme une couche d'orchestration intelligente entre la base documentaire interne et les utilisateurs autorisé

2. Objectifs du Projet

2.1 Objectif Principal

Concevoir et implémenter un système intelligent interne basé sur une architecture RAG sécurisée, intégrant :

- Une ingestion multi-format des documents internes,
- Un stockage structuré des données,
- Une indexation vectorielle sémantique,

- Un contrôle d'accès hiérarchique,
- Un moteur de génération contrôlée via LLM.

L'objectif est d'optimiser la circulation des connaissances internes tout en garantissant la confidentialité et la segmentation des données selon les rôles organisationnels.

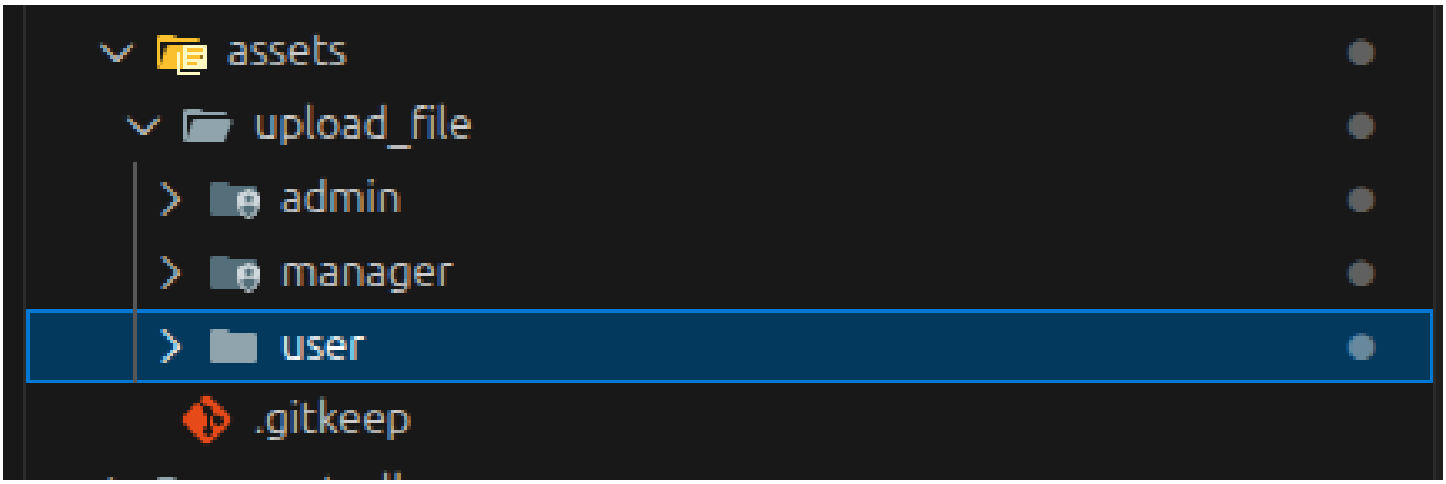
2.2 Objectifs Spécifiques

1. **Ingestion et Structuration des Données**
   Développer un module capable d'ingérer des fichiers PDF, TXT et autres formats internes, puis de les découper en segments exploitables (chunking configurable).
2. **Stockage Persistant et Organisation par Projet Implémenter une base MongoDB pour conserver :**
   - Les métadonnées des fichiers,
   - Les segments de texte,
   - L'association aux projets et aux rôles autorisés.
3. **Indexation Vectorielle et Recherche Sémantique**
   Mettre en place une base vectorielle persistante **(ChromaDB)** afin de permettre une recherche par similarité sémantique, avec filtrage dynamique basé sur les rôles.
4. **Contrôle d'Accès Hiérarchique**
   Implémenter un mécanisme de filtrage au niveau du retrieval garantissant que :
   - Admin accède à toutes les données,
   - Company accède aux données intermédiaires,
   - User accède uniquement aux documents autorisés.
5. **Génération Contrôlée via LLM**
   Configurer un prompt strict afin de :
   - Limiter les hallucinations,
   - Forcer l'utilisation exclusive du contexte récupéré,
   - Structurer les réponses (réponse directe + points clés + sources).
6. **Interface API Structurée**
   Développer une architecture FastAPI modulaire comprenant :
   - Upload et traitement des documents,
   - Gestion des projets,
   - Endpoint de chat sécurisé,
   - Gestion des rôles et authentification.

3. Architecture RAG Hiérarchique Basée sur les Rôles

3.1 Conception Générale

Le système implémente une architecture RAG interne intégrant un modèle hiérarchique à trois niveaux :



- **Admin**
- **Manager**
- **User**

Chaque rôle dispose d'un périmètre d'accès défini.
Le contrôle d'accès n'est pas appliqué après génération, mais directement au niveau du moteur de retrieval sémantique.

Cette approche garantit que la génération du LLM est strictement limitée aux données autorisées pour chaque rôle.

3.2 Intégration du NLP dans le système

Le NLP intervient à deux niveaux fondamentaux :

1. Compréhension sémantique des requêtes utilisateur
   La question en langage naturel est transformée en **embedding vectoriel**.
2. **Génération contrôlée**
   Le **LLM** produit une réponse structurée uniquement à partir du contexte récupéré.

Le système ne repose pas sur une correspondance par mots-clés, mais sur une similarité sémantique dense.

3.3 Pipeline Technique Détaillé

Étape 1 : Segmentation (Chunking)

Les documents internes sont découpés en segments cohérents.

Objectif :

- Optimiser la recherche sémantique
- Respecter la limite de contexte du modèle
- Éviter la dilution de l'information

Chaque chunk est enrichi avec des métadonnées :

```python
class ChunkMetadata(BaseModel):
    source: str
    section: str
    has_code: bool
    allowed_roles: roles

You, 2 days ago | 1 author (You)
class DataChunk(BaseModel):

        _id: Optional[ObjectId]
        file_name: str = Field(..., min_length=1)
        chunk_project_id: str = Field(..., min_length=1)
        chunk_order: int
        chunk_text: str
        chunk_date_pushed : datetime = Field(default_factory=lambda: datetime.now())
        metadata: ChunkMetadata
        model_config = ConfigDict(
        arbitrary_types_allowed=True,
        populate_by_name=True
      )

        @classmethod
        def get_indexes(cls):      You, 2 days ago • ajouter la option indexes …
            return [
                {
                    "key": [("chunk_project_id", 1)],
                    "name": "chunk_project_id_index",
                    "unique": False
                },
                {
                    "key": [("file_name", 1)],
                    "name": "file_name_index",
                    "unique": False
                }
            ]
```

- source
- ordre
- projet
- rôle autorisé

Étape 2 : Vectorisation

```python
class chunkModel(BaseDataModel):
        async def upsert_project_chunks_to_chroma(

                query = {"chunk_project_id": project_id}
                if role:
                        query["metadata.allowed_roles"] = role

                total = 0
                cursor = self.collection.find(query)

                batch = []
                async for doc in cursor:
                        batch.append(doc)
                        if len(batch) >= batch_size:
                                total += self._upsert_batch(vector_store, batch)
                                batch = []

                if batch:
                        total += self._upsert_batch(vector_store, batch)

                return total

        def _upsert_batch(self, vector_store: Chroma, docs: list[dict]) -> int:
                texts = []
                metadatas = []
                ids = []      You, 14 minutes ago • Uncommitted changes

                for d in docs:
                        ids.append(str(d["_id"]))
                        texts.append(d.get("chunk_text", ""))

                        md = d.get("metadata", {}) or {}

                        metadatas.append({
                                "chunk_project_id": str(d.get("chunk_project_id", "")),
                                "allowed_roles": str(md.get("allowed_roles", "")),
                                "source": str(md.get("source", "")),
                                "section": str(md.get("section", "")),
                                "file_name": str(d.get("file_name", "")),
                                "chunk_order": int(d.get("chunk_order", 0)),
                        })

                vector_store.add_texts(texts=texts, metadatas=metadatas, ids=ids)
                return len(docs)
```

Chaque segment est converti en embedding numérique.

Principe mathématique :
Deux textes proches en signification auront une distance cosinus faible dans l'espace vectoriel.

Cela permet de retrouver un passage pertinent même si la formulation diffère totalement.

Étape 3 : Indexation Vectorielle Persistante

Create Index | Refresh | VIEWING INDEXES SEARCH INDEXES

Name & Definition	Type	Size	Usage	Properties	Status
_id_	REGULAR	41.0 kB	1 (since Wed Mar 04 2026)	UNIQUE	READY
_id ↑					
chunk_project_id_index	REGULAR	36.9 kB	0 (since Wed Mar 04 2026)		READY
chunk_project_id ↑					
file_name_index	REGULAR	36.9 kB	0 (since Wed Mar 04 2026)		READY
file_name ↑					

Les embeddings sont stockés dans une base vectorielle persistante.

Chaque entrée contient :

- **le texte du chunk**
- **son embedding**
- **ses métadonnées (notamment allowed_roles)**

La persistance permet :

- stabilité du système
- scalabilité
- redémarrage sans perte d'index

Étape 4 : Retrieval Hiérarchique

Lorsqu'une question est posée :

1. Vectorisation de la requête.
2. Recherche des Top-K segments les plus proches.
3. Application d'un filtre dynamique basé sur le rôle.

Politique d'accès implémentée :

```python
class RAGController:

    async def ask_question(self, query: str, project_id: str ,role: str):

        def allowed_roles(role: str) -> list[str]:
            if role == "admin":
                return ["admin", "manager", "user"]
            if role == "manager":
                return ["manager", "user"]
            return ["user"]

        roles_list = allowed_roles(role)

        chunk_model = await chunkModel.instance(db_client=self.db_client)

        for r in roles_list:
            await chunk_model.upsert_project_chunks_to_chroma(
                vector_store=self.vector_store,
                project_id=str(project_id),
                role=r
            )


        roles_list = allowed_roles(role)

        if len(roles_list) == 1:
            where = {"allowed_roles": roles_list[0]}
        else:
            where = {"$or": [{"allowed_roles": r} for r in roles_list]}



        retriever = self.vector_store.as_retriever(
                    search_kwargs={
                            "k": 5,
                            "filter": where
                        }
                )

        rag_chain = create_retrieval_chain(retriever, self.combine_docs_chain)

        response = await rag_chain.ainvoke({
                    "input": query,
                    "project_id": project_id,
                    "allowed_roles": role
                })
```

- **Admin → accès total**
- **Manager → accès Manager + User**
- **User → accès uniquement User**

Le filtrage est intégré directement dans la requête vectorielle via les métadonnées.

Conséquence importante :
Un utilisateur ne peut pas récupérer un chunk non autorisé, même si celui-ci est sémantiquement pertinent.

Ceci constitue un mécanisme de sécurité structurel intégré au moteur RAG.

3.4 Génération Contrôlée Anti-Hallucination

Une fois les Top-K chunks récupérés, le système construit **un prompt augmenté** contenant :

```python
class RAGController:
    def __init__(self, db_client, settings = settings):
        self.embeddings = GoogleGenerativeAIEmbeddings(
            model="gemini-embedding-001",
            google_api_key=self.settings.KEY_GEMINI
        )

        self.vector_store = Chroma(
            collection_name="rag_docs",
            embedding_function=self.embeddings,
            persist_directory="/app/data/chroma"
        )

        self.llm = ChatGoogleGenerativeAI(
            model="gemini-2.5-flash",
            google_api_key=self.settings.KEY_GEMINI,
            temperature=0.2
        )


        system_prompt = ("""
                You are an AI technical assistant dedicated  exclusively to project "{project_id}".

                STRICT ISOLATION RULE:
                - You MUST use ONLY the provided context.
                - You MUST NOT access or reference data from any other project.
                - If the answer is not found in the context, respond exactly:
                "The information is not available in this project's documents."

                MULTI-LANGUAGE RULE:
                - Detect the language of the user's question.
                - Answer in the same language (Arabic, French, or English).
                - Preserve technical keywords exactly as they appear in the context.

                TECHNICAL FOCUS:
                - Prioritize technical keywords found in the context.
                - When answering, highlight important technical terms using **bold** formatting.
                - If the question relates to architecture, DevOps, infrastructure, security, or bugs
                provide structured answers.

                ADMIN MODE:
                If the question is analytical (e.g., "what are the common issues", "list all project
                "main risks", "system problems"), then:
                - Summarize patterns from the retrieved context.
                - Group similar issues.
```

```python
class RAGController:
    def __init__(self, db_client, settings = settings):
        )
        self.prompt = ChatPromptTemplate.from_messages([
            ("system", system_prompt),
            ("human", "{input}"),
        ])

        self.combine_docs_chain = create_stuff_documents_chain(self.llm, self.prompt)
```

- Instruction système stricte
- Contexte récupéré

- Question utilisateur

Règles imposées au modèle :

- Utiliser uniquement le contexte fourni.
- Ne pas inventer d'informations.
- Indiquer explicitement si la réponse n'existe pas dans le contexte.
- Répondre dans la langue de la question.

Cette configuration réduit fortement le risque d'hallucination.

3.5 Différence avec un RAG Classique

Un RAG standard :

- Se concentre uniquement sur la pertinence sémantique.

Votre système :

- Combine pertinence **sémantique + contrôle hiérarchique**.
- Implémente une **sécurité au niveau du retrieval**.
- Structure la connaissance par projet et rôle.
- Intègre un stockage hybride **(MongoDB + Vector Store**).

Cela positionne le système comme une solution interne d'orchestration intelligente des connaissances d'entreprise.

Analyse Comparative des Résultats selon le Rôle

L'expérimentation comparative démontre le fonctionnement effectif du mécanisme de contrôle hiérarchique intégré au moteur RAG.

Pour une même requête utilisateur, le système génère des réponses différentes selon le rôle authentifié :

- En mode Admin, le moteur de retrieval accède à l'ensemble des segments indexés (Admin + Manager + User). Le contexte récupéré est plus riche et peut inclure des informations sensibles ou stratégiques.
- En mode Manager, la récupération est limitée aux segments autorisés pour Manager et User uniquement. Les données strictement administratives ne sont pas accessibles.
- En mode User, le système restreint strictement la recherche aux segments marqués comme accessibles au niveau User. Toute information supérieure dans la hiérarchie est exclue dès la phase de retrieval.

Il est important de souligner que la restriction ne s'applique pas après génération, mais au moment de la recherche vectorielle.

Cela signifie qu'un chunk non autorisé n'est jamais transmis au modèle de génération, garantissant ainsi une sécurité structurelle du système.

Cette démonstration valide l'intégration d'un RAG hiérarchique sécurisé, où la pertinence sémantique et le contrôle d'accès sont combinés dans une même couche algorithmique.

Impact Sécurité et Gouvernance des Données

L'intégration du filtrage hiérarchique au niveau du retrieval transforme l'architecture RAG en un mécanisme de gouvernance des données.

Contrairement à un contrôle d'accès classique appliqué au niveau applicatif, ici la restriction est intégrée dans la requête vectorielle elle-même. Cela présente plusieurs avantages critiques :

1. **Isolation des données**
   Les segments non autorisés ne sont jamais injectés dans le contexte du LLM. Le modèle n'a donc aucune exposition aux informations restreintes, ce qui élimine le risque de fuite indirecte via génération.
2. **Conformité interne**
   Le système respecte la segmentation organisationnelle **(Admin / Manager / User)**. Cette approche facilite l'alignement avec les politiques internes de sécurité et les exigences d'audit.
3. **Traçabilité**
   Chaque réponse peut être reliée aux segments récupérés et à leurs métadonnées. Cela permet :

- Vérification interne
- Audit des réponses générées
- Analyse post-incident si nécessaire

1. Réduction du risque d'erreur humaine
   Même si un utilisateur tente une requête ambiguë ou volontairement large, le moteur ne dépassera jamais son périmètre d'autorisation.

En conséquence, le système ne constitue pas uniquement un assistant intelligent, mais une couche d'orchestration sécurisée des connaissances internes, combinant :

- Recherche sémantique
- Génération contrôlée
- Gouvernance hiérarchique intégrée

Cette architecture renforce la fiabilité opérationnelle du système dans un contexte d'entreprise technique où la confidentialité et la segmentation des données sont critiques.

4. Analyse Critique des Choix Techniques

4.1 Choix du Chunking

Le découpage en chunks constitue un paramètre critique dans tout système RAG.


```
_id: ObjectId('69a4eb12a4fd21f41d74db1e')
file_name : "Guide_Strat_gique_Entreprises_Cibles_pour_PFE_en_IA___Chatbots__Maroc_…"
chunk_project_id : "69a4eadea4fd21f41d74db1c"
chunk_order : 0
chunk_text : "Guide  Strat é gique  :  Entreprises  Cibles  pour
              PFE  en  IA  & Ch…"
▾ metadata : Object
    source : "Guide_Strat_gique_Entreprises_Cibles_pour_PFE_en_IA___Chatbots__Maroc_…"
    section : "Section 1"
    has_code : false
    allowed_roles : "admin"
```

```
_id: ObjectId('69a4eb12a4fd21f41d74db1f')
file_name : "Guide_Strat_gique_Entreprises_Cibles_pour_PFE_en_IA___Chatbots__Maroc_…"
chunk_project_id : "69a4eadea4fd21f41d74db1c"
chunk_order : 1
chunk_text : "dynam iques au M aroc qui correspondent parfaitem ent à votre expertis…"
▾ metadata : Object
    source : "Guide_Strat_gique_Entreprises_Cibles_pour_PFE_en_IA___Chatbots__Maroc_…"
    section : "Section 2"
    has_code : false
    allowed_roles : "admin"
```


Un chunk trop petit entraîne :

- Perte de contexte
- Réponses fragmentées
- Multiplication inutile des segments

Un chunk trop grand entraîne :

- Bruit sémantique
- Retrieval moins précis
- Risque de dépassement de la fenêtre de contexte

Le choix d'un chunk_size configurable avec overlap représente un compromis optimisé entre précision et cohérence.

L'overlap permet d'éviter la coupure d'une phrase ou d'un raisonnement technique au milieu d'un segment.

4.2 Choix d'un Vector Store Persistant

L'utilisation d'une base vectorielle persistante présente plusieurs avantages :

- Conservation des embeddings après redémarrage
- Scalabilité horizontale possible
- Séparation claire entre stockage brut (MongoDB) et index sémantique (Chroma)

Cette architecture hybride améliore la modularité du système.

4.3 Intégration du Filtrage au Niveau du Retrieval

Beaucoup de systèmes appliquent le contrôle d'accès après génération.

Dans ce projet, le filtrage est appliqué au moment même de la requête vectorielle.

Avantage fondamental :
Le LLM ne voit jamais une donnée non autorisée.

Cela réduit :

- Les risques de fuite d'information
- Les erreurs logiques
- Les incohérences de réponse

5. Limites Techniques du Système

Un rapport sérieux doit toujours mentionner les limites.

5.1 Re-indexation Dynamique

Si l'indexation se fait à la demande, cela peut augmenter la latence.

Optimisation future possible :
Indexer une seule fois lors du processing et non à chaque question.

5.2 Dépendance à la Qualité des Documents

Le système ne corrige pas :

- Les erreurs dans les documents
- Les incohérences internes
- Les documents obsolètes

Un RAG ne peut être plus fiable que sa base documentaire.

5.3 Limite de la Fenêtre de Contexte

Si Top-K est trop élevé :

- Risque de dépassement tokens
- Dilution de la réponse
- Baisse de précision

Le choix de K doit être calibré empiriquement.

5.4 Absence de Reranking Avancé

Le système repose sur similarité cosinus standard.

Amélioration possible :

- Reranker cross-encoder
- Hybrid search (BM25 + vector)

Cela augmenterait la précision dans les cas ambigus.

6. Axes d'Amélioration Avancés

6.1 RAG Hybride

Combiner :

- Recherche lexicale
- Recherche vectorielle

Pour améliorer recall + précision.

6.2 Mémoire Conversationnelle

Actuellement, chaque question peut être traitée indépendamment.

Amélioration :
Intégrer une mémoire contextuelle multi-turn pour des conversations longues.

6.3 Monitoring des Hallucinations

Implémenter :

- Score de confiance
- Détection automatique d'incohérences
- Logging des réponses sensibles

6.4 Multi-Index par Projet

Au lieu d'un seul index global :
Créer un index par projet pour améliorer performance et isolation.

Backend

FastAPI 0.1.0 OAS 3.1

/openapi.json

users

- POST /users Create User
- GET /users/me Me

default

- GET /test-users Test Users
- GET /chain Read Chain
- POST /chat/{id_load} Chat With Docs
- POST /data-load/{id_load} Read Data Load
- POST /proccese/{id_load} Validate Data Load

auth

- POST /auth/login Login

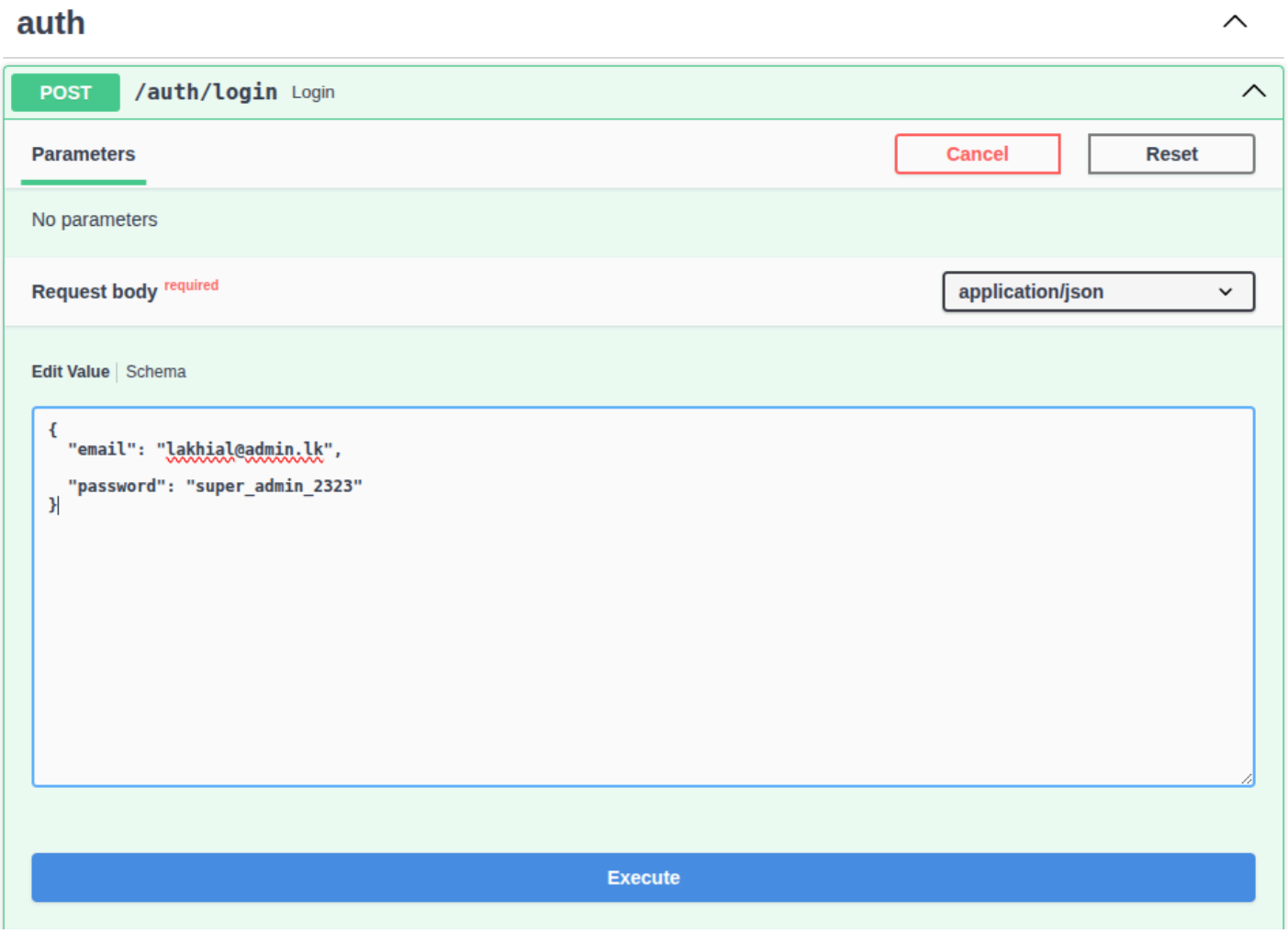
conversations

- GET /conversations List Conversations
- POST /conversations Create Conversation
- GET /conversations/{conversation_id}/messages List Messages
- POST /conversations/{conversation_id}/messages Add User Message

1. Module Authentication (auth)

POST /auth/login

```
_id: ObjectId('69a67c9e7dea2ce331201542')
email : "lakhial@admin.lk"
password_hash : "$pbkdf2-sha256$29000$/b8XYuwdolTq3XsvZYzRmg$agggjEOY5XQx.0IfCdOBwUYqKW…"
role : "admin"
created_by : null
is_active : true
created_at : 2026-03-03T06:15:58.519+00:00
```



Rôle : Authentification des utilisateurs

Cet endpoint permet à un utilisateur (Admin, Manager ou User) de s'authentifier en fournissant :

- email
- password

Si les informations sont valides :

- Le système génère un **JWT** (**JSON Web Token**).
- Le token est retourné avec **token_type = bearer**.

Responses

Curl

```
curl -X 'POST' \
  'http://0.0.0.0:8000/auth/login' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "email": "lakhial@admin.lk",

  "password": "super_admin_2323"
}'
```

Request URL

```
http://0.0.0.0:8000/auth/login
```

Server response

Code	Details
200	Response body

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoiNjlhNjdjOWU3ZGVhMmNlMzMxMjAxNTQyIiwicm9sZSI6ImFkbWluIiwiZXhwIjoxNzcyNTYzMjgyfQ.z9-1ieYrDz3ww54D-2H3LEjmAZ7PXNnJnCCVgZDJOLE",
  "token_type": "bearer"
}
```

Download

Response headers

```
content-length: 216
content-type: application/json
date: Tue,03 Mar 2026 17:41:22 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	*No links*

Media type

application/json

Controls Accept header.

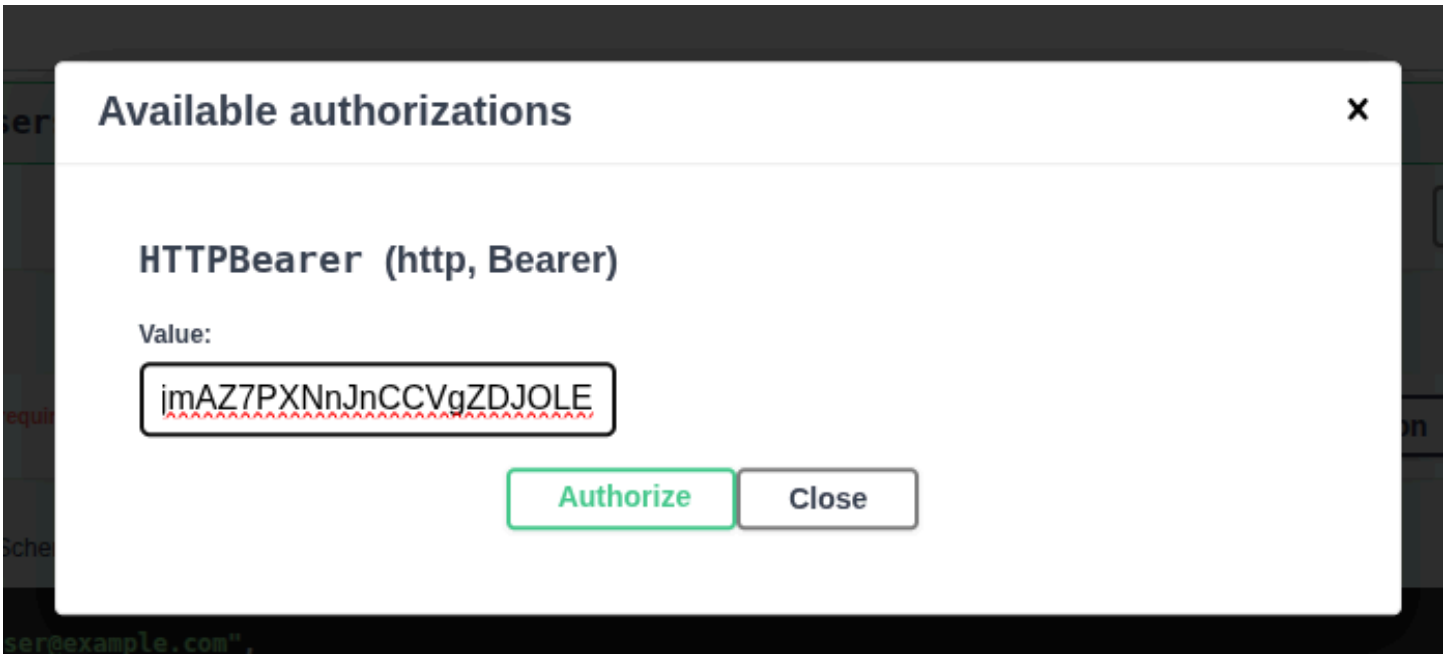
Example Value | Schema

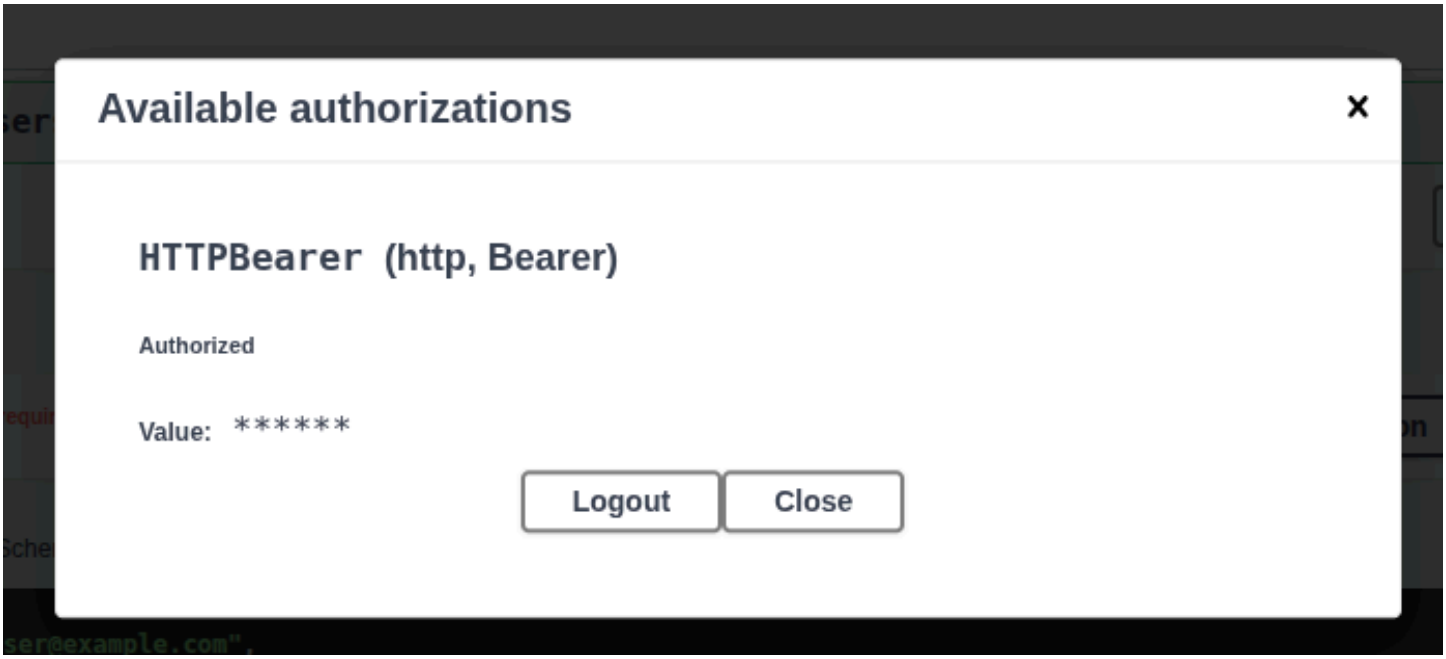
```
{
  "access_token": "string",
  "token_type": "bearer"
}
```

Fonction technique :

- Vérification du mot de passe (hash sécurisé).
- Génération d'un token signé contenant l'identifiant et le rôle.
- Le token sera utilisé pour sécuriser tous les endpoints protégés.

Module Authentication et Gestion du Token





Le système d'authentification repose sur un mécanisme sécurisé basé sur JWT (JSON Web Token). Lorsqu'un utilisateur soumet ses identifiants via l'endpoint POST /auth/login, le backend vérifie les informations, puis génère un access_token signé contenant l'identifiant de l'utilisateur, son rôle (admin, manager ou user) ainsi qu'une date d'expiration.

Après réception du token, celui-ci est copié puis inséré dans Swagger via le bouton Authorize situé en haut de l'interface. Le token est utilisé au format :

Bearer <access_token>

Une fois validé, Swagger inclut automatiquement le header suivant dans toutes les requêtes protégées :

Authorization: Bearer <token>

Ce mécanisme permet :

- La sécurisation des endpoints sensibles.
- L'identification automatique du rôle utilisateur.
- L'activation du contrôle d'accès hiérarchique (RBAC).
- La protection contre les accès non autorisés.

Ainsi, toute requête vers un endpoint protégé sans token valide entraîne une réponse 401 Unauthorized, garantissant l'intégrité du système.

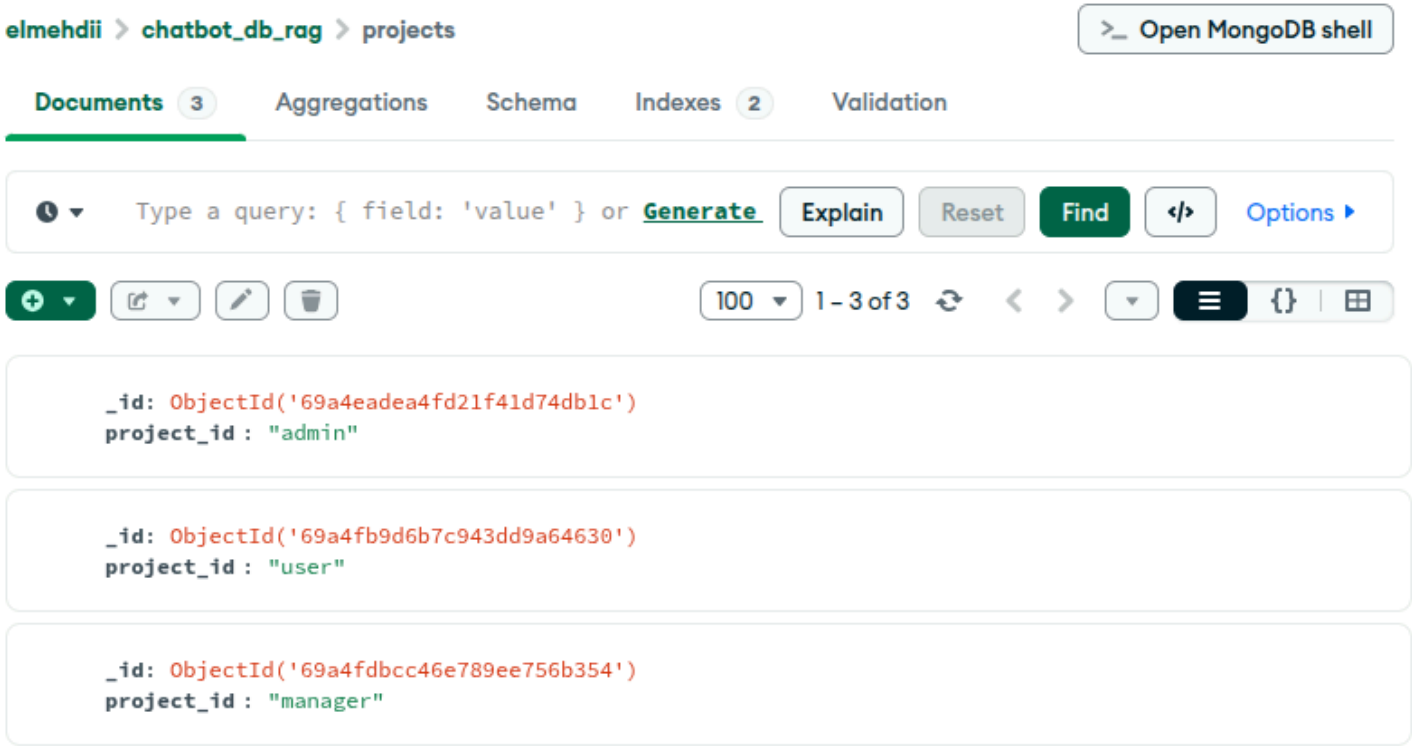
code python:

src > middlewares > auth_guard.py > ...

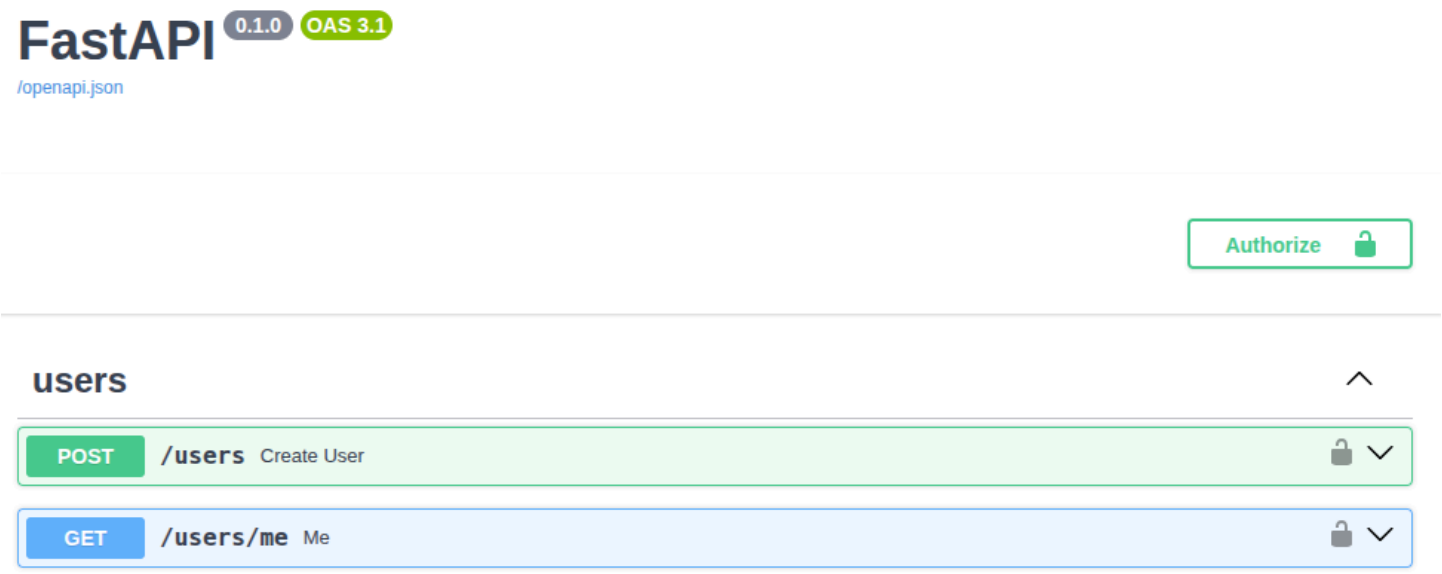
```python
from fastapi import HTTPException, Depends
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from jose import jwt, JWTError
import os

bearer_scheme = HTTPBearer()

SECRET_KEY = os.getenv("JWT_SECRET", "SUPER_SECRET_KEY")
ALGORITHM = "HS256"


def get_current_user(
    credentials: HTTPAuthorizationCredentials = Depends(bearer_scheme)
):
    token = credentials.credentials

    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        return payload
    except JWTError:
        raise HTTPException(status_code=401, detail="Invalid or expired token")
```

2. Module Users

Ce module est responsable de la gestion des comptes utilisateurs et de l'implémentation du modèle de contrôle d'accès basé sur les rôles (RBAC).
Il constitue la base de la hiérarchisation du système **(Admin / Manager / User)**.

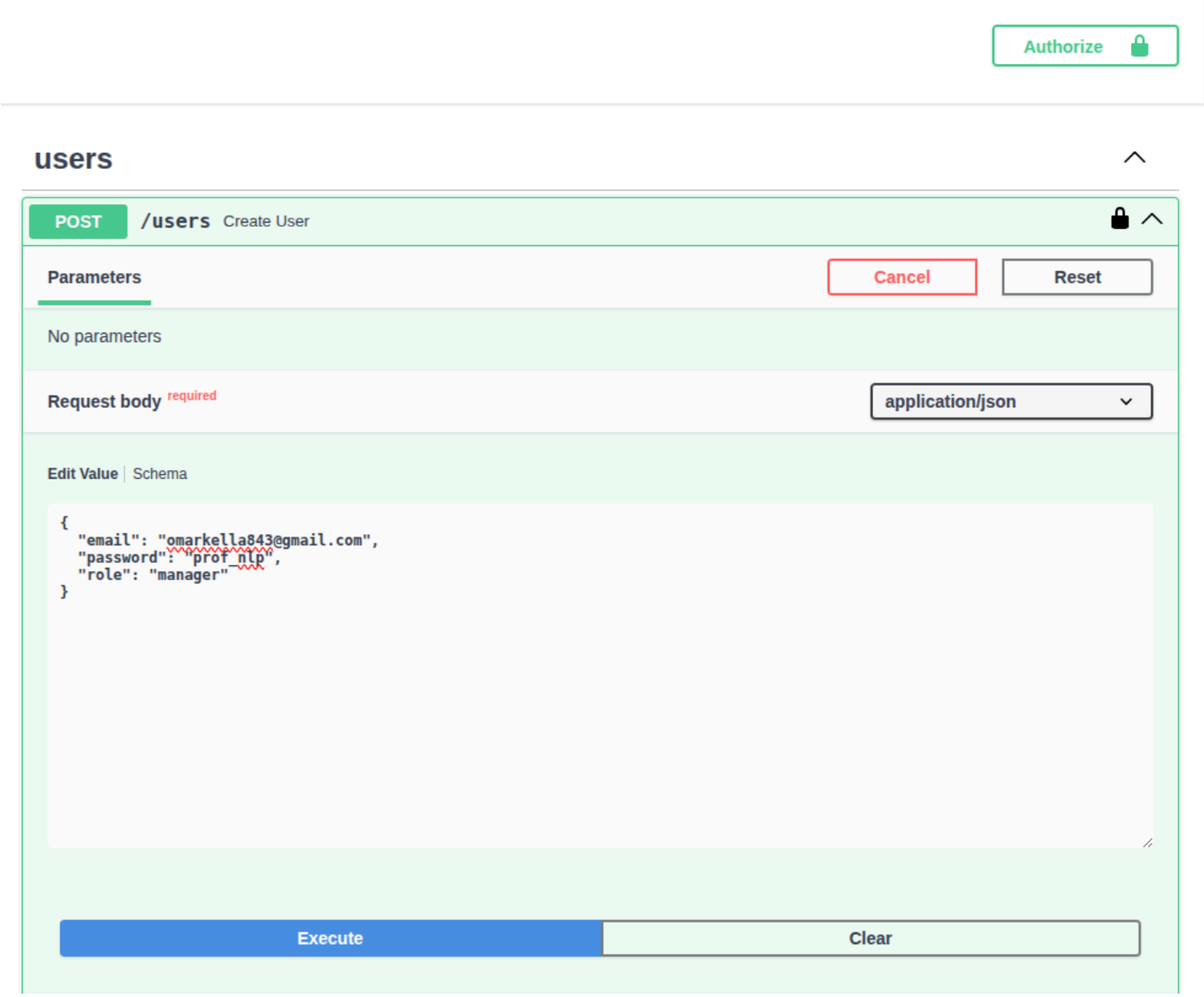


POST /users



Rôle : Création d'un nouvel utilisateur

Cet endpoint permet à un utilisateur autorisé (généralement Admin) de créer un nouveau compte dans le système.



Paramètres requis :

- **email :** identifiant unique de l'utilisateur
- **password :** mot de passe en clair (sera hashé côté serveur)
- **role** : niveau d'autorisation **(admin / manager / user)**

Fonctionnement technique détaillé :

1. Validation des données entrantes
   - Vérification du format email
   - Vérification que le rôle est valide
   - Vérification que l'email n'existe pas déjà

1. Hash sécurisé du mot de passe
   - Le mot de passe n'est jamais stocké en clair.

- Il est transformé via un algorithme de hash sécurisé.
- Cela protège les utilisateurs en cas de fuite de base de données.

1. Création du document utilisateur en base de données


```
_id: ObjectId('69a7217f6d47b6afdb0354d5')
email : "omarkella843@gmail.com"
password_hash : "$pbkdf2-sha256$29000$fC.FcA6BsBbiHMM4ByBEyA$dvwInb5lD//yL7OCAsJkyWwV9E…"
role : "manager"
created_by : "69a67c9e7dea2ce331201542"
is_active : true
created_at : 2026-03-03T17:59:27.066+00:00
```


- **id unique**
- **email**
- **role**
- **is_active**
- **created_at**
- **created_by (id de l'utilisateur ayant effectué la création)**

1. Intégration dans la hiérarchie
   - Le rôle défini influencera directement :
     - Les permissions API
     - L'accès aux documents
     - Le périmètre de retrieval dans le moteur RAG

Objectif fonctionnel

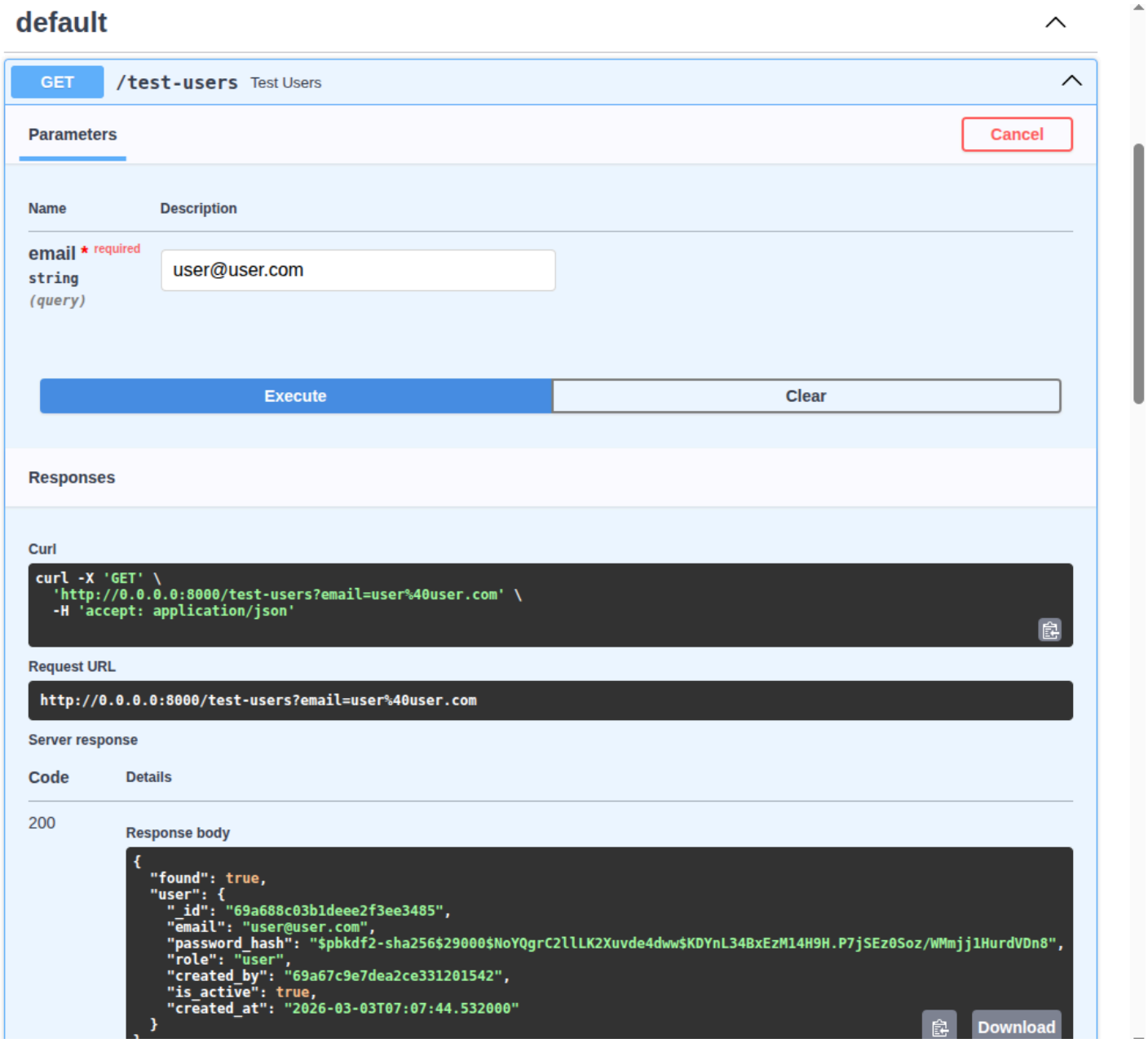
Ce endpoint permet :

- D'organiser la structure hiérarchique interne.
- De segmenter l'accès aux données.
- D'implémenter un contrôle d'accès strict.
- De préparer l'intégration avec le moteur RAG hiérarchique.

Il constitue la première couche de sécurité logique du système.

GET /test-users

Rôle : Vérification de l'existence d'un utilisateur



Cet endpoint permet de vérifier si un utilisateur est bien enregistré dans la base de données.

Fonctionnement technique :

1. Recherche par email dans la base.
2. Si trouvé :
   - Retourne les informations principales.
3. Sinon :
   - Retourne une indication que l'utilisateur n'existe pas.

Objectif technique

Cet endpoint est principalement utilisé pour :

- Tester le bon fonctionnement de la création utilisateur.
- Vérifier l'intégrité des données en base.

- Faciliter le debugging backend.
- Contrôler que la hiérarchie est correctement enregistrée.

Importance dans l'architecture globale

Le module Users ne se limite pas à la gestion des comptes.
Il est fondamental car :

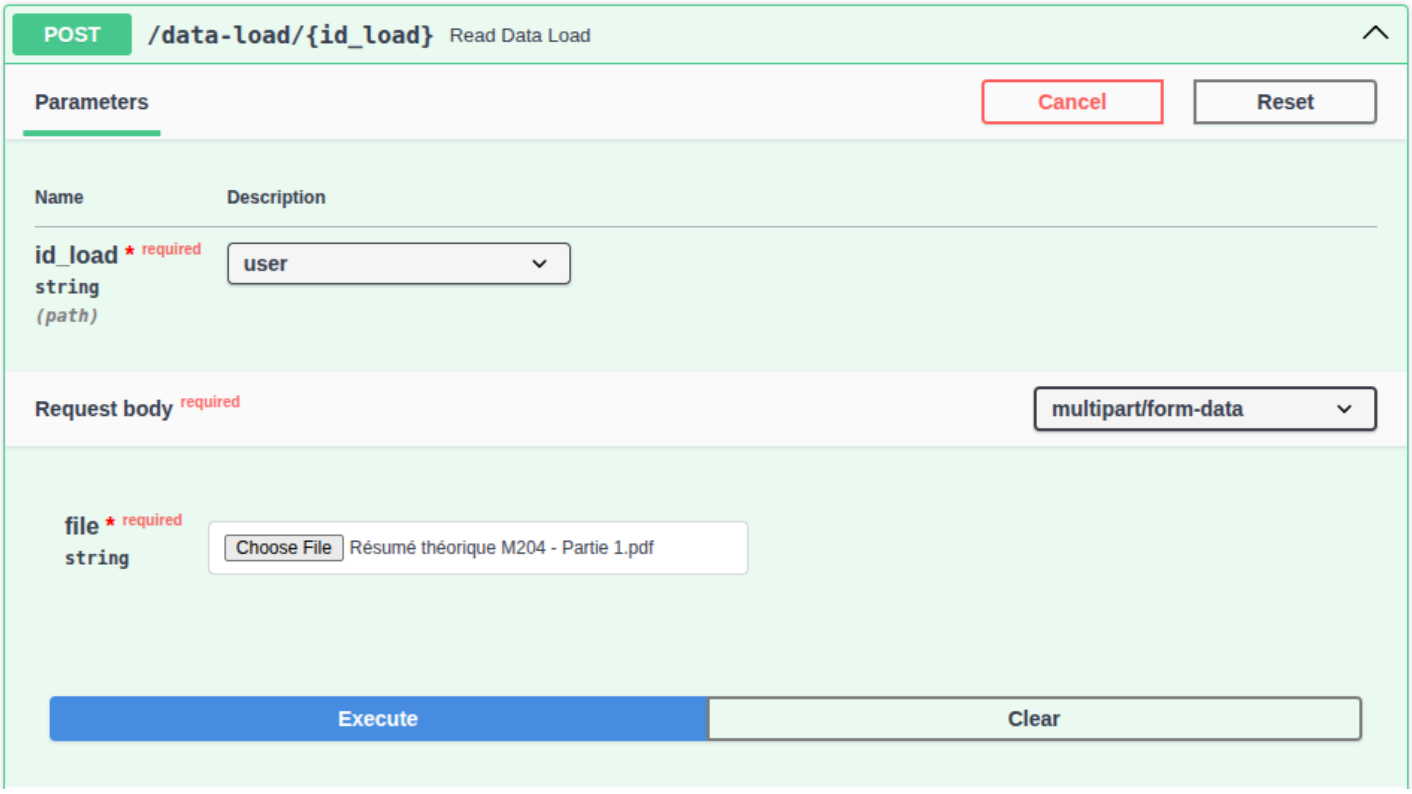
- Il alimente le système d'authentification **(JWT).**
- Il détermine les permissions dynamiques.
- Il conditionne le filtrage hiérarchique dans le moteur **RAG.**
- Il garantit que chaque utilisateur opère dans son périmètre autorisé.

Sans ce module, le contrôle hiérarchique au niveau du retrieval serait impossible.

3. Module Data Ingestion

POST /data-load/{id_load}

Rôle : Upload et structuration initiale des documents selon la hiérarchie



Cet endpoint constitue la première étape critique du pipeline RAG.
Il ne se limite pas à un simple upload de fichier, mais représente la phase d'intégration contrôlée des connaissances dans le système.

Paramètre Clé : id_load

Responses

Curl

```
curl -X 'POST' \
  'http://0.0.0.0:8000/data-load/user' \
  -H 'accept: application/json' \
  -H 'Content-Type: multipart/form-data' \
  -F 'file=@Résumé théorique M204 - Partie 1.pdf;type=application/pdf'
```

Request URL

```
http://0.0.0.0:8000/data-load/user
```

Server response

Code	Details
200	Response body

```
{
  "status": "data load successfully",
  "file_name": "R_sum__th_orique_M204__Partie_1_user_8081.pdf",
  "project_id": "69a4fb9d6b7c943dd9a64630",
  "saved_to_db": true,
  "role": "user",
  "metadata_id": "69a71bce6a844d802dedeef7"
}
```

Download

Response headers

```
content-length: 210
content-type: application/json
date: Tue,03 Mar 2026 17:35:09 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	*No links*

id_load représente le niveau hiérarchique associé au document :

- **admin**
- **manager**
- **user**

Ce choix n'est pas décoratif.
Il détermine dès l'ingestion :

- Le périmètre d'accès futur
- Le scope du projet associé
- Le filtrage appliqué au moteur de retrieval

Autrement dit, la hiérarchie est intégrée dès l'entrée de la donnée.

Type de requête

- multipart/form-data
- Fichier PDF ou format accepté

L’utilisation de multipart/form-data permet :

- Le transfert sécurisé de fichiers volumineux
- La gestion correcte du contenu binaire
- L’extension future vers d’autres formats

Fonctionnement Technique Détaillé

1. Validation du fichier

Le backend effectue :

- Vérification de l’extension
- Vérification de la taille maximale
- Vérification que le rôle est valide

Cette étape évite :

- Upload de fichiers non autorisés
- Risques de sécurité
- Corruption de pipeline

2. Sauvegarde Physique du Fichier

Le fichier est sauvegardé sur le serveur dans un répertoire structuré par rôle.

Objectif :

- Organisation claire des ressources
- Séparation logique des données
- Préparation pour traitement ultérieur

Cette séparation physique reflète la hiérarchie logique.

3. Création ou Association à un Projet

Le système vérifie si un projet existe déjà pour ce rôle.

Si non :

- Création automatique d’un projet

Cela permet :

- Regroupement logique des documents
- Isolation des contextes documentaires
- Scalabilité future par projet

4. Enregistrement des Métadonnées en Base

Un document metadata est créé en base de données contenant :

- **file_name**
- **role associé**
- **project_id**
- **metadata_id**
- **informations temporelles (created_at)**
- **éventuellement created_by**

Cette étape est fondamentale.

Pourquoi ?

Parce que le moteur RAG ne travaille pas directement sur les fichiers bruts.
Il travaille sur des entités documentaires référencées et structurées.

Importance Architecturale

Ce module représente la couche de fondation du système.

Sans ingestion structurée :

- Le chunking serait désorganisé
- Le filtrage hiérarchique serait impossible
- Le retrieval ne pourrait pas appliquer de contrôle d'accès

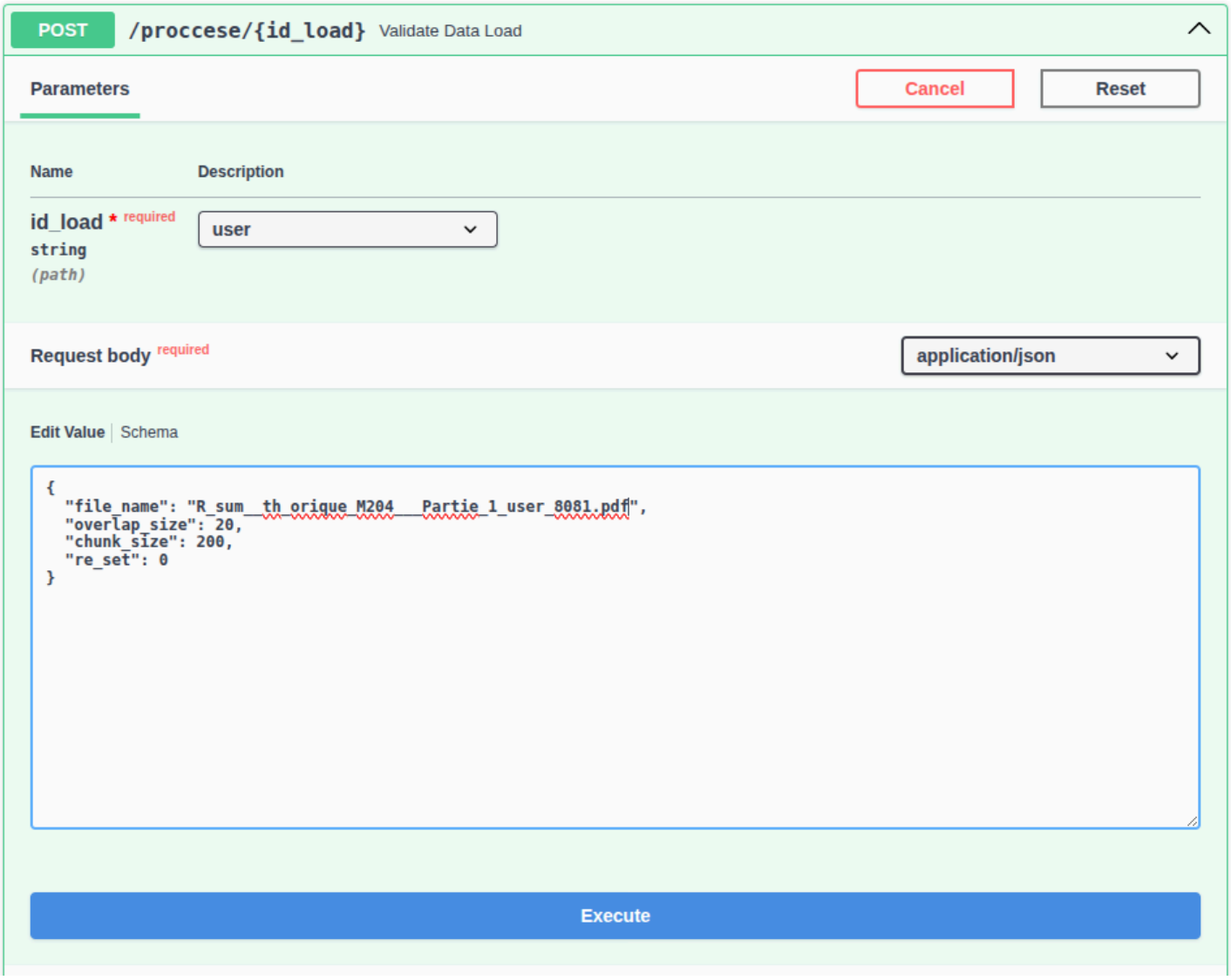
Ce design permet :

1. Séparation claire entre stockage brut et traitement NLP.
2. Intégration native de la hiérarchie dès l'entrée des données.
3. Traçabilité complète de chaque document.
4. Scalabilité vers une architecture multi-projets.

En résumé, ce module transforme un simple fichier uploadé en ressource structurée prête à entrer dans le pipeline RAG sécurisé.

4. Module Processing (Prétraitement NLP)

POST /proccese/{id_load}



Rôle : Extraction, normalisation et segmentation sémantique du document

Ce module constitue la phase centrale du pipeline NLP.
Après l'ingestion physique du fichier, cette étape transforme le document brut en unités textuelles exploitables par un moteur RAG.

Il s'agit d'une transformation structurelle et non d'un simple traitement textuel.

Paramètres d'entrée

- **file_name :** nom du fichier à traiter
- **chunk_size :** taille maximale d'un segment textuel
- **overlap_size :** nombre de caractères/tokens partagés entre deux segments consécutifs
- **re_set :** indique si les anciens chunks doivent être supprimés et recréés

Ces paramètres rendent le système configurable et adaptable selon la nature des documents (techniques, longs, structurés, etc.).

Fonctionnement Technique Détaillé

1. Extraction du texte

Le backend lit le fichier PDF et extrait le contenu textuel brut.

Cette phase peut inclure :

- Lecture page par page
- Nettoyage des caractères parasites
- Normalisation des sauts de ligne

Objectif :
Transformer un document binaire en texte exploitable.

2. Segmentation (Chunking)

Le texte complet est découpé en segments appelés “chunks”.

Pourquoi segmenter ?

Les modèles LLM ont :

- Une limite de fenêtre de contexte
- Une efficacité optimale sur des passages cohérents mais courts

Un document entier ne peut pas être injecté tel quel dans un LLM.

Le chunking permet :

- D’isoler des unités sémantiquement cohérentes
- D’optimiser la recherche vectorielle
- D’améliorer la précision du retrieval

3. Application de l’Overlap

L’overlap permet de répéter une partie du texte entre deux chunks consécutifs.

Pourquoi c’est important ?

Sans overlap :

- Une phrase peut être coupée en deux
- Le contexte peut être fragmenté
- Le retrieval peut perdre de la précision

Avec overlap :

- La continuité logique est préservée
- Le modèle reçoit un contexte plus stable
- La cohérence des réponses est améliorée

C’est un choix stratégique fondamental dans la qualité d’un système RAG.

4. Enrichissement avec Métadonnées

Chaque chunk créé est stocké avec des métadonnées structurées :

```
_id: ObjectId('69a5e59f8a57134358b610e7')
file_name : "Guide_Strat_gique_Entreprises_Cibles_pour_PFE_en_IA___Chatbots__Maroc_…"
chunk_project_id : "69a4fb9d6b7c943dd9a64630"
chunk_order : 14
chunk_text : "Chatbots de ven
              Assistants virtue
              e-com m erce, La
              pages intelligent
              X…"
▾ metadata : Object
    source : "Guide_Strat_gique_Entreprises_Cibles_pour_PFE_en_IA___Chatbots__Maroc_…"
    section : "Section 15"
    has_code : false
    allowed_roles : "user"
```

```
_id: ObjectId('69a5e59f8a57134358b610e8')
file_name : "Guide_Strat_gique_Entreprises_Cibles_pour_PFE_en_IA___Chatbots__Maroc_…"
chunk_project_id : "69a4fb9d6b7c943dd9a64630"
chunk_order : 15
chunk_text : "CRM , Reporting
              intelligent, Chat
              d'autom atisatio
              Adyl Agency Autom …"
▾ metadata : Object
    source : "Guide_Strat_gique_Entreprises_Cibles_pour_PFE_en_IA___Chatbots__Maroc_…"
    section : "Section 16"
    has_code : false
    allowed_roles : "user"
```

```
_id: ObjectId('69a5e59f8a57134358b610e9')
file_name : "Guide_Strat_gique_Entreprises_Cibles_pour_PFE_en_IA___Chatbots__Maroc_…"
chunk_project_id : "69a4fb9d6b7c943dd9a64630"
```

- source : fichier d'origine
- rôle autorisé : admin / manager / user
- ordre du segment
- project_id
- informations de traçabilité

Ces métadonnées jouent un rôle critique car elles permettent :

- Le filtrage hiérarchique lors du retrieval
- L'identification des sources dans la réponse finale
- La gouvernance documentaire

Résultat

Responses

Curl

```
curl -X 'POST' \
  'http://0.0.0.0:8000/proccese/user' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "file_name": "R_sum__th_orique_M204___Partie_1_user_8081.pdf",
  "overlap_size": 20,
  "chunk_size": 200,
  "re_set": 0
}'
```

Request URL

```
http://0.0.0.0:8000/proccese/user
```

Server response

Code	Details
200	Response body

```
{
  "status": "Chunks created: \n 514  \nConst.procceing_file_success"
}
```

Download

Response headers

```
content-length: 67
content-type: application/json
date: Tue,03 Mar 2026 17:37:52 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	*No links*

Media type

application/json

Controls Accept header.

Exemple de retour API :

"Chunks created: 514"

Cela signifie que le document a été transformé en 514 unités exploitables, chacune prête à être vectorisée et indexée.

Objectif NLP Global

Transformer un document non structuré en :

- Un ensemble structuré de segments cohérents
- Annotés avec des métadonnées hiérarchiques
- Optimisés pour la vectorisation
- Compatibles avec la recherche sémantique

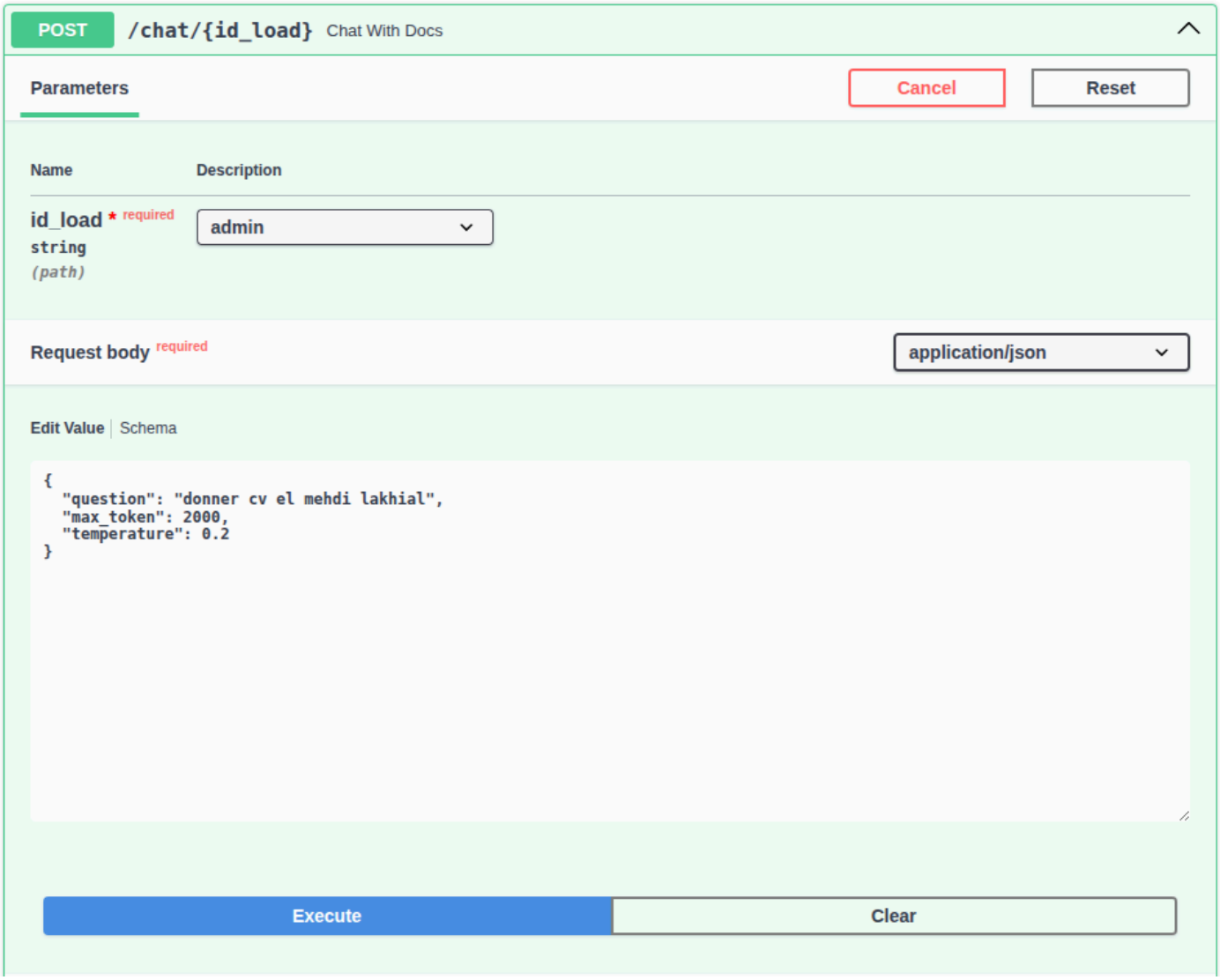
Cette étape constitue le pont entre :

Stockage brut
et
Intelligence artificielle conversationnelle.

Sans ce module, le moteur RAG ne pourrait pas fonctionner correctement.

5. Module Chat (RAG Engine)

POST /chat/{id_load}



Rôle : Orchestration intelligente entre recherche sémantique et génération contrôlée

Ce module constitue le cœur du système.
Il combine :

- Recherche vectorielle
- Filtrage hiérarchique
- Génération via LLM
- Structuration contrôlée de la réponse

Contrairement à un chatbot classique, ce module ne génère jamais une réponse à partir de zéro. Il s'appuie exclusivement sur la base documentaire interne.

Responses

Curl

```
curl -X 'POST' \
  'http://0.0.0.0:8000/chat/admin' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "question": "donner cv el mehdi lakhial",
  "max_token": 2000,
  "temperature": 0.2
}'
```

Request URL

```
http://0.0.0.0:8000/chat/admin
```

Server response

Code	Details
200	

Response body


```
{
  "allowed_roles": "admin",
  "question": "donner cv el mehdi lakhial",
  "answer": "Voici les informations disponibles concernant El Mehdi Lakhial dans les documents du projet :\n\n
1) Direct Answer\nEl Mehdi Lakhial est associé au **PROJET DE SESSION NLP** intitulé \"Chatbot Assistant Servic
e Client Intelligent basé sur le RAG (Retrieval-Augmented Generation)\".\n\n2) Key Technical Points\n*   **Rôl
e/Compétences** : **Generative AI**, **NLP**, **Full-Stack**.\n*   **Projet** : Chatbot Assistant Service Clien
t Intelligent basé sur le **RAG** (Retrieval-Augmented Generation).\n*   **Date** : 2025/02/04.\n\n3) Source Ev
idence\n*   \"PROJET DE SESSION NLP Chatbot Assistant Service Client Intelligent basé sur le RAG (Retrieval-Aug
mented Generation) Generative AI / NLP / Full-Stack : El Mehdi Lakhial 2025/02/04\""
}
```


Download

Response headers

```
content-length: 818
content-type: application/json
date: Tue,03 Mar 2026 17:29:04 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	*No links*

Media type

application/json

Controls Accept header.

Example Value | Schema

request :

Voici le même contenu sans aucun marqueur Markdown :

El Mehdi Lakhial est associé au projet intitulé :
PROJET DE SESSION NLP – Chatbot Assistant Service Client Intelligent basé sur le RAG (Retrieval-Augmented Generation).

Informations principales :

Direct Answer
El Mehdi Lakhial participe au projet NLP nommé "Chatbot Assistant Service Client Intelligent basé sur le RAG".

Key Technical Points
Rôle et compétences : Generative AI, NLP, Full-Stack.
Projet : Développement d'un chatbot intelligent basé sur la technique RAG (Retrieval-Augmented Generation).
Date associée au projet : 04/02/2025.

Source Evidence
PROJET DE SESSION NLP
Chatbot Assistant Service Client Intelligent basé sur le RAG (Retrieval-Augmented Generation)
Generative AI / NLP / Full-Stack : El Mehdi Lakhial
Date : 2025/02/04

Niveau de certitude : élevé.

Paramètres d'entrée

- question : requête en langage naturel
- max_token : limite maximale de génération
- temperature : contrôle du niveau de créativité du modèle

Ces paramètres permettent d'adapter le comportement du modèle :

- temperature faible → réponses plus déterministes
- max_token contrôlé → maîtrise des coûts et du contexte

Fonctionnement Interne Détaillé

1. Vectorisation de la Question

La question est transformée en embedding numérique dense.

Objectif :

Représenter le sens de la phrase dans un espace vectoriel multidimensionnel.

Cela permet une comparaison mathématique avec les chunks indexés.

2. Recherche Sémantique (Cosine Similarity)

Le système calcule la similarité cosinus entre :

- Le vecteur de la question
- Les vecteurs des chunks

Formule simplifiée :

similarité = (A · B) / (||A|| × ||B||)

Plus le score est proche de 1, plus le sens est similaire.

Le système sélectionne les **Top-K** chunks les plus proches.

3. Application du Filtrage Hiérarchique

Avant même la génération, un filtre dynamique est appliqué sur les métadonnées.

Politique implémentée :

- Admin → accès à tous les chunks
- Manager → accès aux chunks manager + user
- User → accès uniquement aux chunks user

Point fondamental :

Les chunks non autorisés ne sont jamais envoyés au modèle.

Cela signifie :

- Aucune exposition indirecte
- Aucune fuite par génération
- Sécurité intégrée au moteur RAG

C'est une différence majeure avec un contrôle d'accès classique appliqué après génération.

4. Construction du Prompt Augmenté

Le système construit un prompt structuré contenant :

1. Instruction système stricte
2. Contexte récupéré (Top-K chunks)
3. Question utilisateur

Exemple logique :

"Réponds uniquement à partir du contexte fourni. Si l'information n'existe pas, indique-le explicitement."

Cela réduit fortement les hallucinations.

5. Génération via LLM

Le modèle génère une réponse :

- Basée uniquement sur le contexte

- Structurée
- Adaptée à la langue de la question

La température est contrôlée pour limiter la créativité excessive.

Structure de la Réponse

La réponse renvoyée par l'API contient :

- allowed_roles : rôle utilisé pour la requête
- question : question originale
- answer structuré en :

1. Direct Answer
   Réponse synthétique et précise.
2. Key Technical Points
   Résumé structuré des éléments importants.
3. Source Evidence
   Référence aux segments utilisés.

Point Technique Fondamental

Le contrôle d'accès est appliqué au niveau du retrieval et non après la génération.

Cela implique que :

- Le LLM ne reçoit jamais une information non autorisée.
- La sécurité est intégrée dans la couche algorithmique.
- La hiérarchie influence directement la construction du contexte.

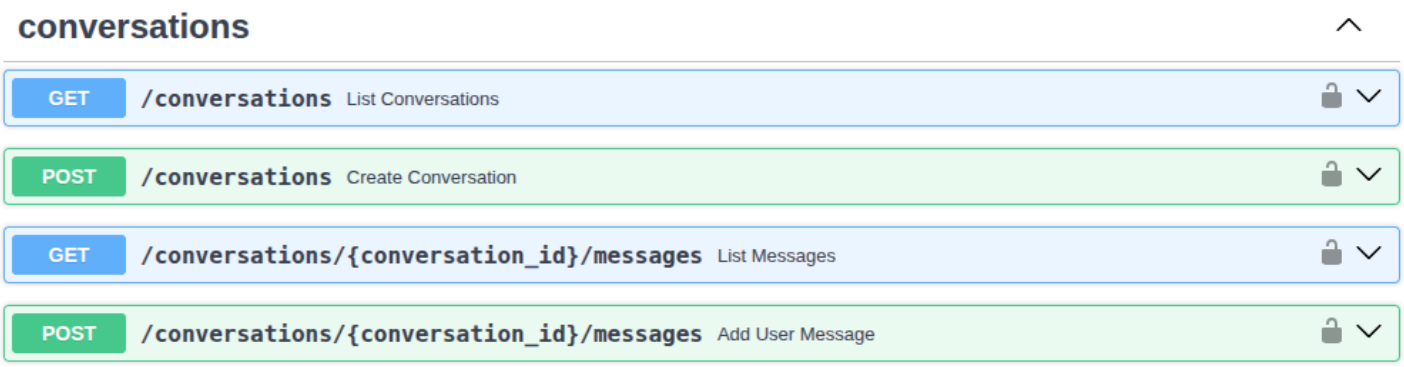
Ce design transforme le module Chat en un moteur RAG hiérarchique sécurisé adapté aux environnements d'entreprise.

6. Module Conversations

Ce module assure la gestion de l'historique conversationnel et représente une couche essentielle pour la traçabilité et l'analyse des interactions utilisateurs.

Contrairement à un simple chat stateless, le système conserve un contexte structuré par session.

POST /conversations



Rôle : Création d'une nouvelle session de discussion

Cet endpoint permet de créer une nouvelle conversation associée à l'utilisateur authentifié.

Fonction technique :

- **Génération d'un identifiant unique (conversation_id).**
- **Association de la conversation à l'utilisateur via user_id.**
- **Enregistrement d'un titre ou métadonnées si nécessaire.**
- **Initialisation du contexte conversationnel.**

Objectif :

Structurer les échanges et éviter un mélange de requêtes dans un seul flux global.

Cela permet d'avoir plusieurs discussions parallèles organisées.

GET /conversations

GET /conversations/{conversation_id}/messages List Messages

Parameters

Cancel

Name	Description
conversation_id * required string (path)	69a8979358cacda25dab0de7

Execute | Clear

Responses

Curl

```
curl -X 'GET' \
  'http://0.0.0.0:8000/conversations/69a8979358cacda25dab0de7/messages' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoiNjlhNjdjOWU3ZGVhMmNlMzMxMjAxNTQyIiwicm9
```

Request URL

```
http://0.0.0.0:8000/conversations/69a8979358cacda25dab0de7/messages
```

Server response

Code Details

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/conversations' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoiNjlhNjdjOWU3ZGVhMmNlMzMxMjAxNTQyIiwicm9sZSI6ImFkbWluIiwiZXhwIjoxNzcyNjYyNzYxfQ.cq5npct0QdQG4oV9MlHZii25yIcAb
```

Request URL

```
http://127.0.0.1:8000/conversations
```

Server response

Code Details

200

Response body


```
{
  "items": [
    {
      "id": "69a8a392d94bc190a0365ae5",
      "title": "test convesation et chunk database",
      "updated_at": "2026-03-04T21:36:19.934000",
      "created_at": "2026-03-04T21:26:42.831000"
    },
    {
      "id": "69a8979358cacda25dab0de7",
      "title": "test conversations rag ",
      "updated_at": "2026-03-04T20:48:35.465000",
      "created_at": "2026-03-04T20:35:31.944000"
    }
  ]
}
```

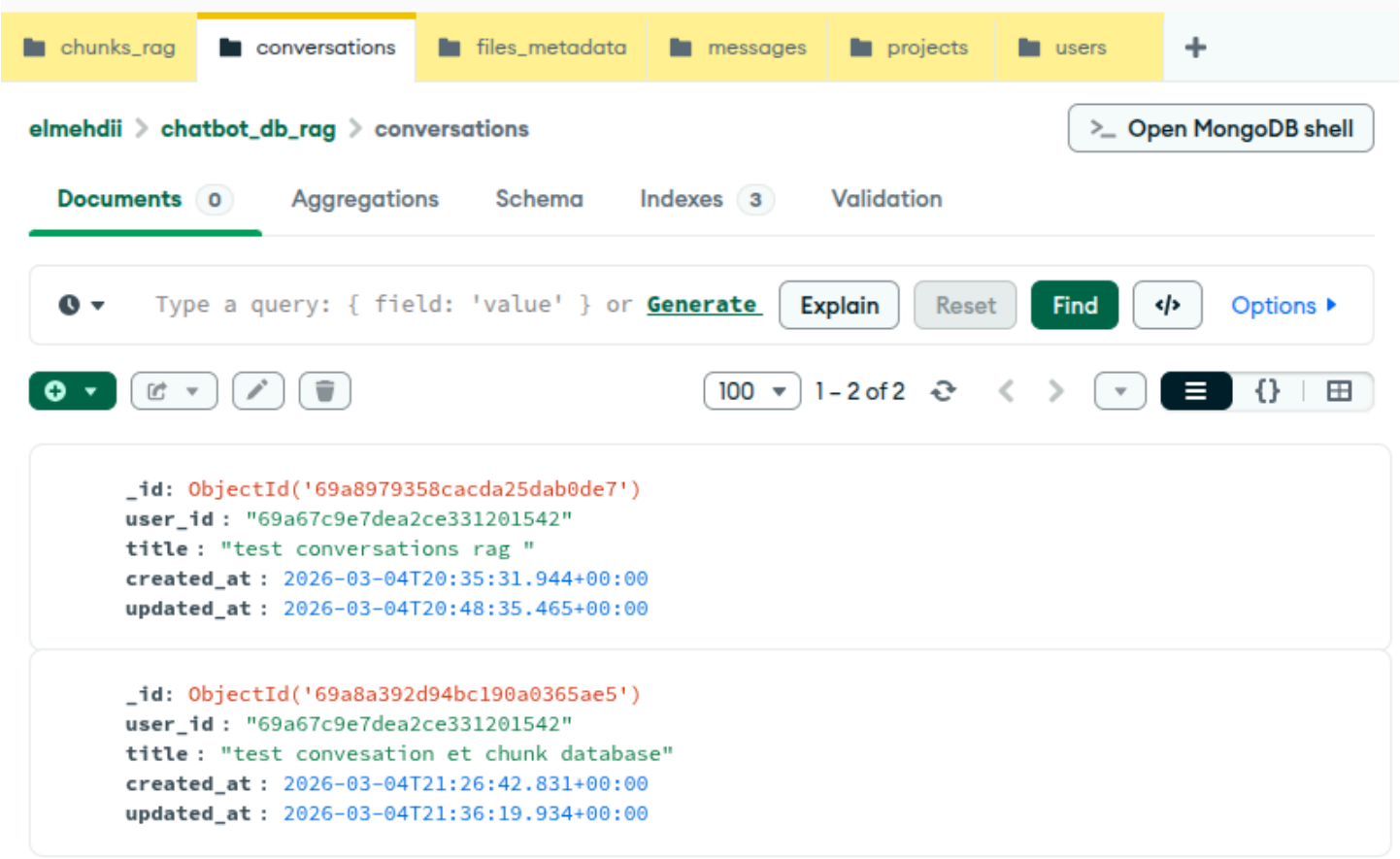

Download

Response headers

```
content-length: 326
content-type: application/json
date: Wed,04 Mar 2026 21:50:19 GMT
server: uvicorn
```

Rôle : Liste des conversations de l'utilisateur

Retourne toutes les sessions associées à l'utilisateur connecté.



Fonction technique :

- Requête filtrée par **user_id.**
- Retour des métadonnées principales **(id, titre, date)**.

Objectif :

Permettre la navigation dans l'historique des discussions.

Important pour :

- Reprendre une conversation existante.
- Organiser les interactions par sujet.
- Améliorer l'expérience utilisateur.

GET /conversations/{conversation_id}/messages

GET /conversations/{conversation_id}/messages List Messages

Parameters

Cancel

Name	Description
conversation_id * required string (path)	69a8979358cacda25dab0de7

Execute | Clear

Responses

Curl

```
curl -X 'GET' \
  'http://0.0.0.0:8000/conversations/69a8979358cacda25dab0de7/messages' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoiNjlhNjdjOWU3ZGVhMmNlMzMxMjAxNTQyIiwicm9
```

Request URL

```
http://0.0.0.0:8000/conversations/69a8979358cacda25dab0de7/messages
```

Server response

Code Details

Rôle : Récupération des messages d'une conversation spécifique

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/conversations/69a8979358cacda25dab0de7/messages' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoiNjlhNjdjOWU3ZGVhMmNlMzMxMjAxNTQyIiwicm9sZSI6ImFkbWluIiwiZXhwIjoxNzcyNjYyNzYxfQ.cq5npct0QdQG4oV9MlHZii25yIcAb
```

Request URL

```
http://127.0.0.1:8000/conversations/69a8979358cacda25dab0de7/messages
```

Server response

Code Details

200

Response body


```
{
  "items": [
    {
      "id": "69a8985058cacda25dab0de8",
      "role": "admin",
      "content": "bonjour donner project el mehdi lakhial de npl",
      "sources": null
    },
    {
      "id": "69a8985458cacda25dab0de9",
      "role": "assent",
      "content": "Bonjour,\n\nLe projet d'El Mehdi Lakhial est un **Chatbot Assistant Service Client Intelligent basé sur le RAG (Retrieval-Augmented Generation)**.\n\n**Point
s Techniques Clés :**\n*   **Generative AI**\n*   **NLP**\n*   **Full-Stack**\n*   Date : **2025/02/04**\n\n**Source Evidence :**\n\"PROJET DE SESSION NLP\nChatbot Assistant S
ervice Client Intelligent basé sur le RAG\n(Retrieval-Augmented Generation)\nGenerative AI / NLP / Full-Stack :\nEl Mehdi Lakhial\n2025/02/04\"",
      "sources": null
    },
    {
      "id": "69a898e658cacda25dab0dea",
      "role": "admin",
      "content": "donnee les projet de el mehdi lakhial ",
      "sources": null
    },
    {
      "id": "69a898e858cacda25dab0deb",
      "role": "assent",
      "content": "Voici le projet associé à El Mehdi Lakhial :\n\n1) **PROJET DE SESSION NLP**\n\n2) Key Technical Points:\n*   **Chatbot Assistant Service Clie
*\n*   Basé sur le **RAG** (Retrieval-Augmented Generation)\n*   Technologies: **Generative AI / NLP / Full-Stack**\n\n3) Source Evidence:\n*   \"**PROJET DE SE
tbot Assistant Service Client Intelligent basé sur le **RAG** (Retrieval-Augmented Generation) **Generative AI / NLP / Full-Stack** : **El Mehdi Lakhial** 2025/02/04\"
```


Download

Retourne l'ensemble des messages liés à une session donnée.

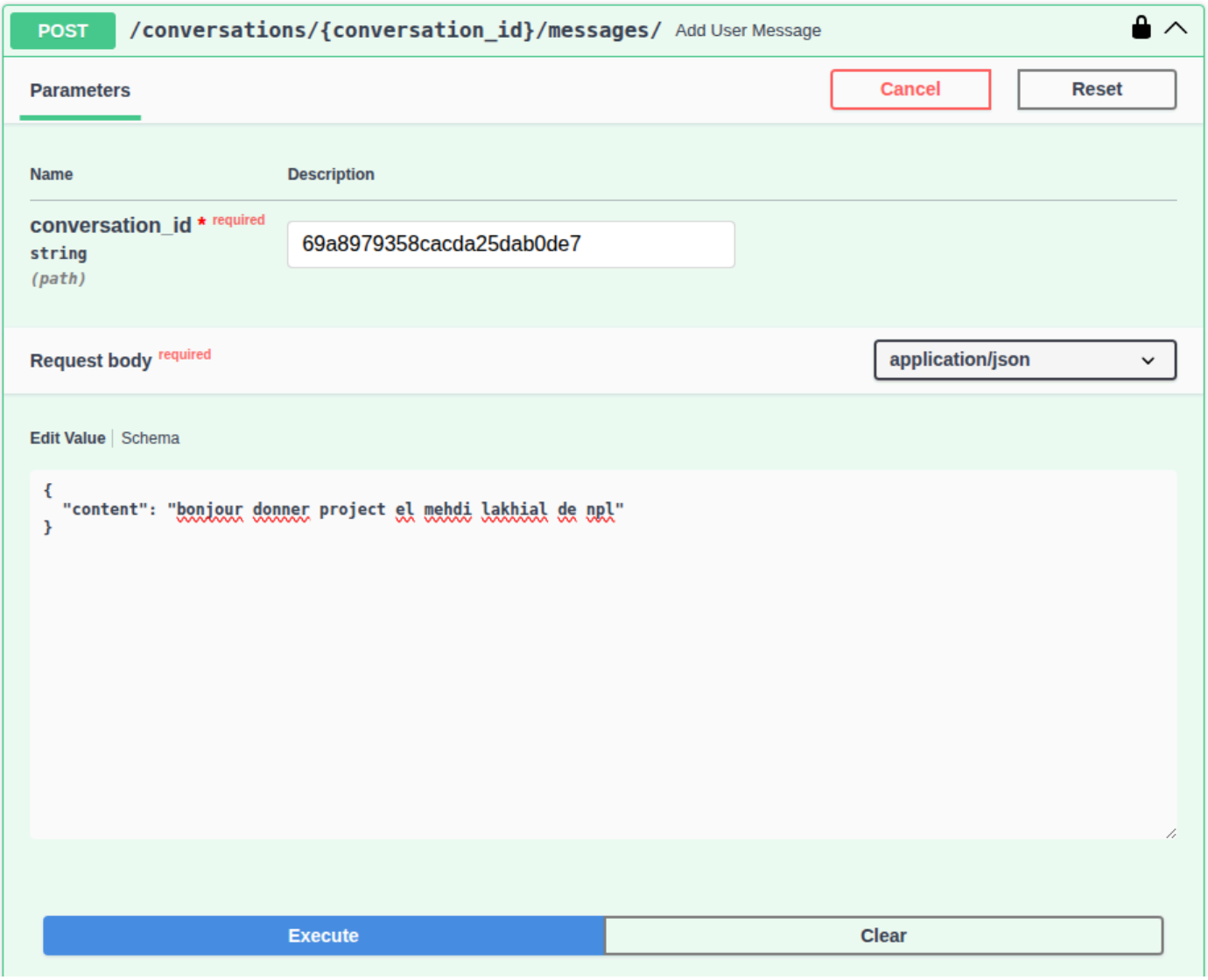
Fonction technique :

- **Filtrage par conversation_id.**
- **Tri chronologique.**
- **Retour structuré des messages (user / assistant).**

Objectif :

- Maintenir la continuité du dialogue.
- Permettre l'analyse des réponses générées.
- Faciliter l'audit interne.

POST /conversations/{conversation_id}/messages



Rôle : Enregistrement d'un nouveau message

Cet endpoint permet d'ajouter un message utilisateur ou système dans une conversation existante.

Fonction technique :

Responses

Curl

```
curl -X 'POST' \
  'http://0.0.0.0:8000/conversations/69a8979358cacda25dab0de7/messages/' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoiNjlhNjdjOWU3ZGVhMmNlMzMxMjAxNTQyIiwicm9
  -H 'Content-Type: application/json' \
  -d '{
  "content": "bonjour donner project el mehdi lakhial de npl"
}'
```

Request URL

```
http://0.0.0.0:8000/conversations/69a8979358cacda25dab0de7/messages/
```

Server response

Code	Details
200	Response body

```
{
  "answer": "Bonjour,\n\nLe projet d'El Mehdi Lakhial est un **Chatbot Assistant Service Client Intelligent basé sur le RAG (Retrieval-Augmented Generation)**.\n\n**Points Techniques Clés :**\n*   **Generative AI**\n*   **NLP**\n*   **Full-Stack**\n*   Date : **2025/02/04**\n\n**Source Evidence :**\n\"PROJET DE SESSION NLP\nChatbot Assistant Service Client Intelligent basé sur le RAG\n(Retrieval-Augmented Generation)\nGenerative AI / NLP / Full-Stack :\nEl Mehdi Lakhial\n2025/02/04\""
}
```

Download

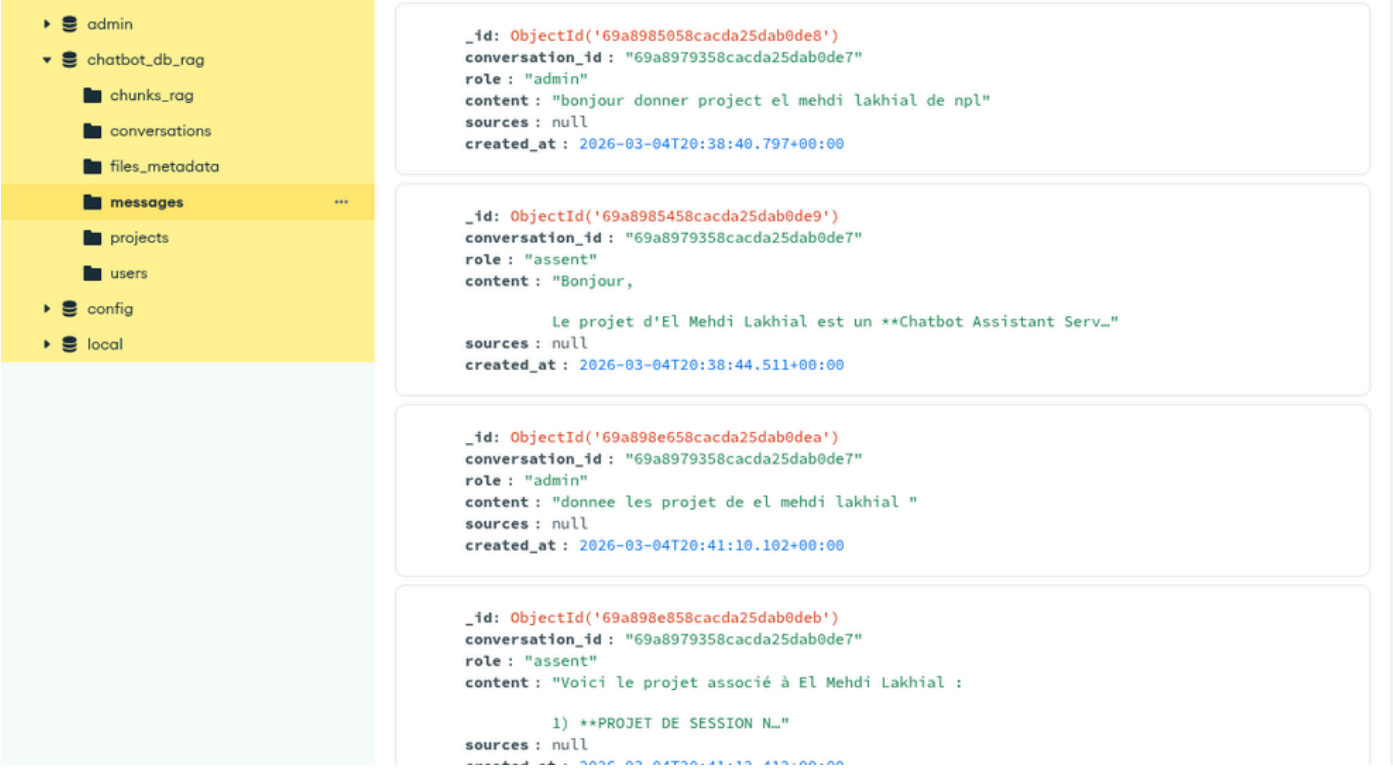
Response headers

```
content-length: 491
content-type: application/json
date: Wed,04 Mar 2026 20:38:40 GMT
server: uvicorn
```

Responses

Code	Description	Links

- **Enregistrement du message avec :**



  - **sender (user / assistant)**
  - **contenu**

    - **timestamp**
    - **conversation_id**
- Conservation en base de données.

Objectif :

Garantir la persistance des interactions pour :

- Analyse comportementale
- Optimisation du moteur RAG
- Détection de questions fréquentes
- Debugging technique

Importance Technique du Module Conversations

Ce module apporte une valeur stratégique au système :

1. Traçabilité
   Chaque réponse générée peut être reliée à une requête spécifique.
2. Audit interne
   Possibilité de vérifier :
   - Ce que l'utilisateur a demandé.
   - Ce que le système a répondu.
   - Si la réponse respecte les règles hiérarchiques.
3. Amélioration continue
   L'analyse des logs permet :
   - Identifier les requêtes fréquentes.
   - Détecter les faiblesses du retrieval.
   - Ajuster les paramètres (chunk_size, Top-K, temperature).
4. Supervision en environnement entreprise
   Permet un contrôle global du comportement du système.

Ce module transforme le chatbot en système supervisable et exploitable à grande échelle.

Résumé Technique Global du Backend

Le backend implémente une architecture complète orientée entreprise combinant sécurité, hiérarchisation et intelligence artificielle.

Il comprend :

1. Authentification sécurisée via JWT
   - Signature des tokens
   - Gestion des rôles
   - Protection des endpoints

2. Gestion hiérarchique des utilisateurs
    - Admin / Manager / User
    - RBAC intégré au niveau applicatif
3. Ingestion documentaire segmentée
    - Upload structuré par rôle
    - Organisation par projet
    - Métadonnées traçables
4. Pipeline NLP complet
    - Extraction du texte
    - Segmentation intelligente (chunking avec overlap)
    - Génération d'embeddings
    - Indexation vectorielle persistante
5. Moteur RAG hiérarchique sécurisé
    - Recherche par similarité cosinus
    - Filtrage dynamique selon le rôle
    - Génération contrôlée anti-hallucination
6. Persistance des conversations
    - Historique structuré
    - Audit
    - Amélioration continue

Vision Architecturale

L'architecture backend repose sur trois principes fondamentaux :

- Sécurité intégrée à tous les niveaux
- Segmentation hiérarchique native
- Orchestration intelligente de la connaissance interne

Ce système ne se limite pas à un chatbot.
Il constitue une plateforme interne d'accès sémantique sécurisé à la documentation organisationnelle.